

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«САРАТОВСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ Н. Г. ЧЕРНЫШЕВСКОГО»**

УТВЕРЖДАЮ

Зав.кафедрой,

доцент, к. ф.-м. н.

_____ С. В. Миронов

ОТЧЕТ О ПРАКТИКЕ

студентки 3 курса 351 группы факультета КНиИТ

Винник Маргариты Дмитриевны

вид практики: учебная

кафедра: математической кибернетики и компьютерных наук

курс: 3

семестр: 6

продолжительность: 2 нед., с 08.05.2026 г. по 26.05.2026 г.

Руководитель практики от университета,

доцент, к. ф.-м. н.

М. И. Сафрончик

Руководитель практики от организации (учреждения, предприятия),

доцент, к. ф.-м. н.

М. И. Сафрончик

Тема практики: «Модули в языке Python»

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 Теоретическая часть	6
1.1 Библиотека NumPy	6
1.2 Библиотека Pandas	13
1.3 Библиотека Matplotlib / Seaborn	19
1.4 Библиотека Scikit-learn	19
ЗАКЛЮЧЕНИЕ	19

ВВЕДЕНИЕ

Сам по себе «чистый» Python, безусловно, удобен, но для сложных численных расчётов, анализа данных (Data Science) и машинного обучения (Machine learning) его встроенных возможностей недостаточно. Ключевая причина, почему Python стал лидером в области анализа данных и искусственного интеллекта — это его способность легко интегрироваться с кодом на других языках.

Ядро интерпретатора CPython (самая распространённая реализация Python) написано на C и C++. Этот фундамент обеспечивает эффективное управление памятью, работу с объектами и выполнение базовых операций. Многие встроенные функции (print, len) и модули стандартной библиотеки (math, sys) также реализованы на C для достижения максимальной производительности.

Однако настоящая мощь раскрывается в сторонних библиотеках. Такие «тяжеловесы», как NumPy для векторных вычислений, Pandas для анализа табличных данных, а также фреймворки глубокого обучения PyTorch и TensorFlow, являются модулями расширения на C/C++.

Эта же философия расширения лежит в основе библиотек для классического машинного обучения и визуализации. Библиотека scikit-learn, построенная поверх NumPy и SciPy, содержит оптимизированные реализации алгоритмов на C/C++ (например, для вычисления расстояний, построения деревьев решений), предоставляя при этом единый Python-интерфейс.

Для задач визуализации данных фундаментальная библиотека matplotlib наследует возможности векторной графики из C++-библиотек (в частности, Anti-Grain Geometry), а высокоуровневая надстройка seaborn позволяет создавать сложные статистические графики минимальным количеством кода, полагаясь на ту же C/C++-основу.

Таким образом, каждая из ключевых библиотек экосистемы Data Science следует принципу "Python-обёртка над высокопроизводительным ядром".

В этом контексте Python выступает в роли высокоуровневого «клея»: он предоставляет удобный и гибкий интерфейс для сборки и управления высокопроизводительными вычислительными ядрами, написанными на компилируемых языках. Такой подход сочетает простоту разработки со скоростью выполнения кода, что критически важно для задач анализа данных и машинного обучения.

Важно отметить, что все эти библиотеки спроектированы для бесшов-

ной интеграции друг с другом. NumPy обеспечивает единый формат данных (ndarray), который понимают Pandas, SciPy, Scikit-learn и Matplotlib. Pandas, в свою очередь, построен поверх NumPy и добавляет удобную работу с табличными данными и временными рядами. Scikit-learn принимает на вход как NumPy-массивы, так и Pandas-DataFrame. Seaborn изначально задуман для работы с DataFrame и интегрируется с Matplotlib для тонкой настройки графиков. Эта взаимная совместимость создаёт цельную экосистему, где каждая библиотека усиливает возможности другой, оставаясь при этом частью единого "языкового клея" — Python.

1 Теоретическая часть

О чём собственно я буду рассказывать:

В этой работе я рассмотрю несколько основных библиотек-инструментов (модулями на самом деле их уже давно не называет из-за их объёмности и сложности) для специалиста анализа данных и инженера машинного обучения. Вот он - так называемый джентельметский набор, который я постепенно рассмотрю подробнее, объясню что для чего нужно и как использовать:

1. NumPy (Numerical Python);
2. Pandas;
3. Scikit-learn;
4. Matplotlib / Seaborn;

1.1 Библиотека NumPy

По классике, начнём с NumPy, но немного издалека, а именно с типа данных, который имеют объекты, созданные при помощи этой библиотекой. Тип данных **ndarray** библиотеки NumPy представляет собой многомерный массив элементов одного типа. Элементы массива NumPy индексируются кортежем целых неотрицательных чисел.

Массивы NumPy похожи на списки (list) Python или на массивы из встроенного модуля **array**, которые используются для создания плотных массивов данных одного типа. Однако массивы NumPy требуют меньше памяти и обычно работают быстрее, поскольку используют оптимизированный, прекомпилированный код на языке C.

В то время как объект **array** в языке Python обеспечивает только эффективное хранение данных в формате массива, библиотека NumPy добавляет ещё и возможность выполнения эффективных операций с этими данными (будут рассмотрены дальше).

Ключевая особенность — поддержка поэлементных операций. Это позволяет выполнять основные арифметические вычисления сразу над всем массивом, делая код компактным и удобочитаемым. Также доступна поэлементная операция над двумя массивами одинаковой размерности, в результате которой получается другой массив той же размерности, где каждый элемент i, j является результатом вычисления, выполненного над элементами i, j двух исходных массивов.

Таблица 1.1 – Сравнение типов данных для хранения массивов в Python

Характеристика	Встроенный list	Встроенный array из модуля array	ndarray из NumPy
Типы данных	Любые, разнородные	Однородный (один тип)	Однородный (один тип)
Размер	Динамический	Фиксированный	Фиксированный (при создании)
Производительность	Низкая	Средняя	Высокая (благодаря C/C++ ядру)
Функциональность	Базовая (добавление, удаление)	Очень базовая	Огромная (векторизация, линейная алгебра, статистика)
Потребление памяти	Большое	Маленькое	Маленькое (плотное хранение)

Прежде чем начать работать с этой библиотекой, её нужно установить, т.к. она не является встроенным модулем, как, например, модули `math`, `array` и другие стандартные. Установка NumPy происходит по команде в терминале (VS Code, PyCharm): **`pip install numpy`**. Импортируем пакет NumPy под псевдонимом **`np`** (это уже в рабочем файле `.py`):

```
import numpy as np
```

Дальше в примерах кода буду опускать строку с импортом.

Итак, способы создания массивов NumPy (буду сразу приводить примеры кода и давать пояснения):

- Массив NumPy можно создать из данных одного или нескольких списков. Для создания массивов из списков можно воспользоваться функцией **`np.array`**.

```
my_arr = np.array([1, 4, 2, 5, 3])
```

Если мы захотим вывести такой массив, то увидим в консоле следующее: **`[1 4 2 5 3]`**.

С помощью функции **`type()`** можем убедиться, что тип созданного объекта **`ndarray`**. В консоли увидим: **`<class 'numpy.ndarray'>`**.

Сразу рассмотрим пример создания многомерного массива из нескольких "листов":

Для каждого сотрудника компании существует список выплат базового оклада за последние три месяца. Чтобы собрать всю информацию о зарплате в одну структуру данных, можно использовать следующий код:

```
jeff_salary = [2700, 3000, 3000]
nick_salary = [2600, 2800, 2800]
tom_salary = [2300, 2500, 2500]
base_salary = np.array([jeff_salary, nick_salary, tom_salary])
print(base_salary)
```

В консоли это будет выглядеть так:

```
[[2700 3000 3000]
 [2600 2800 2800]
 [2300 2500 2500]]
```

- Явное описание многомерного массива (матрицы). Вот один из способов инициализации многомерного массива с помощью списка списков:

```
my_arr = np.array([range(i, i + 3) for i in [2, 4, 6]])
print(my_arr)
```

В консоли это будет выглядеть так:

```
[[2 3 4]
 [4 5 6]
 [6 7 8]]
```

- Создание массивов из нулей или единиц заданной размерности:

```
my_arr1 = np.zeros((3, 5), dtype=int)
my_arr2 = np.ones((7, 3), dtype=float)

print(my_arr1)
print(my_arr2)
```

Выведет двумерный массив целых нулей:

```
[[0 0 0 0 0]
 [0 0 0 0 0]
 [0 0 0 0 0]]
```

И массив единиц с плавающей точкой:


```
[[1. 1. 1.]  
 [1. 1. 1.]  
 [1. 1. 1.]  
 [1. 1. 1.]  
 [1. 1. 1.]  
 [1. 1. 1.]  
 [1. 1. 1.]]
```

Сразу возникает вопрос: "Зачем это нужно и кто это использует?".

Напомню, что привычный список в Python - это, по сути, массив указателей. Каждый элемент списка - это отдельный полноценный объект в памяти (со своей метайнформацией, типом и счетчиком ссылок), и эти объекты могут быть разбросаны по оперативной памяти как угодно. Чтобы пройти по списку, процессору нужно постоянно "прыгать" по памяти, собирая данные.

Массив в NumPy (ndarray) устроен иначе. Он хранит данные единым, непрерывным блоком в памяти (как массивы в языке C). Кроме того, было уже отмечено, что все элементы в массиве NumPy имеют один и тот же тип данных. Процессор быстро обрабатывает подобные структуры: он может загрузить огромный кусок массива в свой быстрый кэш и обработать его за доли секунды. А еще почти вся математика внутри NumPy написана на высокооптимизированном C, что позволяет обходить медленный интерпретатор Python.

Маленький вывод: В NumPy выделение памяти под массивы работает как в C, т.е. выделяется непрерывный блок в памяти нужного размера. В сочетании с единым типом данных для всех элементов массива и реализаций математических вычислений на C "под капотом" получается скомпенсировать (обойти) медленный интерпретатор Python и получить быстрые вычисления матриц.

- Есть ещё много разнообразных способов создать массив NumPy. Сейчас я их кратко продемонстрирую:

```
# Создаем массив заданной размерности, заполненный  
# заданным числом  
arr_this_full = np.full((3, 5), 98.2)
```

```

# Создаем массив, заполненный линейной последовательностью,
# начинающейся с 0 и заканчивающейся 20, с шагом 2
# (действует подобно встроенной функции range())
arr_this_arange = np.arange(0, 20, 2)

# Создаем массив с пятью значениями, располагающимися
# через равные интервалы в диапазоне между 0 и 1
arr_this_linspace = np.linspace(0, 1, 5)

# Создаем массив заданной размерности равномерно
# распределенных случайных значений от 0 до 1
arr_this_random = np.random.random((3, 3))

# Создаем массив заданной размерности
# нормально распределенных случайных значений
# со средним 0 и стандартным отклонением 1
arr_this_random_normal = np.random.normal(0, 1, (3, 3))

# Создаем массив заданной размерности случайных целых чисел
# в интервале [0, 10)
arr_this_randint = np.random.randint(0, 10, (3, 3))

# Создаем единичную матрицу заданной размерности
arr_this_ones_matrix = np.eye(5)

# Создаем неинициализированный массив из заданного числа
# целочисленных значений. Значениями будут произвольные
# данные, случайно оказавшиеся в соответствующих
# ячейках памяти
arr_this_empty = np.empty((4, 3))

```

Вот как это будет выглядеть в консоли:

```

print(arr_this_full)
[[98.2 98.2 98.2 98.2 98.2]
 [98.2 98.2 98.2 98.2 98.2]

```

```

[98.2 98.2 98.2 98.2 98.2]]

    print(arr_this_arange)
[ 0 2 4 6 8 10 12 14 16 18]

    print(arr_this_linspace)
[0. 0.25 0.5 0.75 1. ]

    print(arr_this_random)
[[0.85842966 0.19472009 0.53980876]
 [0.80552811 0.14153011 0.70063978]
 [0.54145708 0.56197047 0.79364311]]

    print(arr_this_random_normal)
[[ 1.3439026 0.97320026 -0.39531868]
 [ 0.92342143 0.7090699 -0.42076108]
 [-0.21460648 -0.23570425 -0.87608365]]

    print(arr_this_randint)
[[0 2 1]
 [4 4 9]
 [7 8 7]]

    print(arr_this_ones_matrix)
[[1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 1.]]

    print(arr_this_empty)
[[6.95198423e-310 6.95198425e-310 6.95198424e-310]
 [6.95198423e-310 6.95198423e-310 6.95198426e-310]
 [6.95198426e-310 6.95198425e-310 6.95198425e-310]
 [6.95198425e-310 6.95198426e-310 6.95198424e-310]]

```

Массивы NumPy содержат значения простых типов, поэтому важно знать и понимать возможности и ограничения этих типов, хотя вообще-то их немало и даже предоставлена возможность работать с комплексными числами.

Таблица 1.2 – Стандартные типы данных библиотеки NumPy

Тип дан-ных	Описание
bool_	Булев тип (True или False), хранящийся как байт
int_	Тип целочисленного значения по умолчанию (аналогичен типу long в C; обычно int64 или int32)
intc	Идентичен типу int в C (обычно int32 или int64)
intp	Целочисленное значение, используемое для индексов (аналогично типу 'ssize_t' в C; обычно int32 или int64)
int8	Байт (от -128 до 127)
int16	Целое число (от -32 768 до 32 767)
int32	Целое число (от -2 147 483 648 до 2 147 483 647)
int64	Целое число (от -9 223 372 036 854 775 808 до 9 223 372 036 854 775 807)
uint8	Целое число без знака (от 0 до 255)
uint16	Целое число без знака (от 0 до 65535)
uint32	Целое число без знака (от 0 до 4 294 967 295)
uint64	Целое число без знака (от 0 до 18 446 744 073 709 551 615)
float_	Сокращение для названия типа 'float64'
float16	Число с плавающей точкой с половинной точностью: 1 бит знака, 5 битов порядка, 10 битов мантиссы
float32	Число с плавающей точкой с одинарной точностью: 1 бит знака, 8 битов порядка, 23 бита мантиссы
float64	Число с плавающей точкой с удвоенной точностью: 1 бит знака, 11 битов порядка, 52 бита мантиссы
complex_	Сокращение для названия типа 'complex128'
complex64	Комплексное число, представленное двумя 32-битными числами с плавающей точкой
complex128	Комплексное число, представленное двумя 64-битными числами с плавающей точкой

Перейдём к простейшим операциям над массивами NumPy:

- получение атрибутов массивов — определение размера, формы, объема занимаемой памяти и типов данных массивов:

У каждого массива есть атрибуты **ndim** (число размерностей), **shape** (размер каждой размерности) и **size** (общий размер массива):

- индексация массивов — получение и изменение значений отдельных элементов массивов;
- получение срезов массивов — получение и изменение значений подмассивов внутри большого массива;
- изменение формы массивов — изменение формы заданного массива;
- объединение и разбиение массивов — объединение нескольких массивов в один и разделение одного массива на несколько.

1.2 Библиотека Pandas

Логичным продолжением будет рассказать про библиотеку Pandas, т.к. данная библиотека построена поверх NumPy. Pandas (название происходит от эконометрического термина "PAnel DAta панельные данные, т.е. многомерные структурированные наборы информации) - это библиотека с открытым исходным кодом для языка Python, которая предоставляет высокопроизводительные и удобные инструменты для работы с данными.

Pandas — это хорошая, а главное — масштабируемая замена привычному многим Excel. В отличие от табличного процессора, Pandas позволяет обрабатывать файлы объёмом в миллионы строк и автоматизировать процесс очистки и преобразования данных.

На практике редко приходится работать с идеально чистыми данными. Чаще в руки попадают "грязные" с пропусками, дубликатами, выбросами (сильно отклоняющиеся значение от общей массы значений). При работе с данными Pandas позволяет:

1. Читать данные в разных форматах: csv, txt, SQL-базы данных, JSON, HTML-страницы и т.д.

Чаще всего работают с csv-файлами, выгрузками из баз данных или JSON-файлами.

2. Чистить данные, находить пропущенные значения, убирать дубликаты, исправлять ошибки в формате дат или текста.
3. Менять структуру таблиц, создавать новые, в общем придавать данным ту структуру, которая нам требуется.

Основными структурами данных в Pandas являются:

- **Series** (Серия) — это одномерный массив данных, у которого есть метки

(индекс). Представьте себе Series как один столбец в таблице Excel. Он имеет значения (числа, строки, даты) и номера строк слева (индекс).

- **DataFrame** (Датафрейм) — это двумерная табличная структура данных (строки и столбцы). При загрузке таблицы из Excel в Python, она превращается именно в DataFrame.

Установка Pandas происходит по команде в терминале (VS Code, PyCharm):
pip install pandas. Импортируем пакет Pandas под псевдонимом **pd** (это уже в рабочем файле .py):

```
import pandas as pd
```

Дальше в примерах кода буду опускать строку с импортом.

Важное замечание!

Не просто так повествование начиналось с библиотеки NumPy. Для ускорения манипуляций с данными будет использоваться векторизация. **Векторизация — это способность выполнять операции над всем массивом данных сразу, без явного написания цикла.** Если приходится писать цикл *for* для обработки данных в таблице, скорее всего, вы делаете что-то не так. Ищите векторное решение.

Ещё одно важное и удобное отличие Pandas от обычных списков или массивов NumPy — это индексы, но не в привычном нам понимании. В обычном списке элементы доступны только по их порядковому номеру (0, 1, 2...). В pandas у каждой строки есть метка (label), которая называется индекс, т.е. идентификатор данных. Он может быть числом, строкой (например, дата, имя человека, тикер акции) или даже временем.

Одно из свойств - Alignment (выравнивание), позволяющее производить операции над двумя разными объектами pandas. Библиотека автоматически сопоставляет (выравнивает) данные по индексам, а не по порядку следования.

Пример: у вас есть данные о продажах яблок и бананов в двух магазинах.

```
shop1 = {'Яблоки': 10, 'Бананы': 5}
shop2 = {'Бананы': 8, 'Яблоки': 3}
#обратите внимание, порядок другой!
```

Если мы сложим эти данные как обычные списки $[10, 5] + [8, 3]$, мы получим бессмыслицу (сложим яблоки с бананами). Pandas же при сложении

посмотрит на метки: Найдёт "Яблоки" в первом наборе и "Яблоки" во втором, получим $10 + 3 = 13$. Найдёт "Бананы" в первом и "Бананы" во втором, получим $5 + 8 = 13$.

При работе с реальными, неупорядоченными данными имеют значение только метки (индексы), т.е. можно не упорядочивать данные.

Маленький вывод:

- Векторизация: Мы не пишем циклы, мы оперируем целыми столбцами.
- Индексы: Это не просто нумерация строк, это способ связывать данные между собой. Pandas автоматически выравнивает данные по этим меткам.

Вернёмся к структурам Pandas и как с ними работать. Структуру **Series** можно создать двумя способами - из списков и из словарей.

В первом случае мы передаём в конструктор **pd.Series()** обычный список (list). При этом конструктор автоматически создаёт числовой индекс для наших данных.

```
data_list = [10, 20, 30, 40]
# Преобразуем в Series
series_from_list = pd.Series(data_list)
print(series_from_list)
```

Увидим в консоли:

```
0 10
1 20
2 30
3 40
dtype: int64
```

Конструктор **pd.Series()** также даёт возможность задать индексы вручную, передав в параметр **index** список, соответствующих нашим данным, меток.

```
# Данные - продажи
sales = [150, 200, 100]
# Индекс - месяцы
months = ['Jan', 'Feb', 'Mar']
# Явное указание индекса
s = pd.Series(sales, index=months)
print(s)
```

Увидим в консоли:

```
Jan 150
Feb 200
Mar 100
dtype: int64
```

Во втором случае всё ещё проще, т.к. словарь представляет собой пару "ключ - значение"/, то ключи словаря становятся индексом(ами), а значения словаря становятся данными.

```
# Словарь: Валюта -> Курс
currencies = {
    'USD': 92.5,
    'EUR': 99.1,
    'CNY': 12.8
}
# Pandas автоматически поймет, что ключи - это индексы
s_curr = pd.Series(currencies)
print(s_curr)
```

Увидим в консоли:

```
USD 92.5
EUR 99.1
CNY 12.8
dtype: float64
```

Кроме того Series есть полезный атрибут **name**. Это имя всей серии (будущее название колонки, если мы объединим несколько серий в таблицу).

С числовыми метками может возникнуть следующая проблема: возникает путаница и ошибки при задании явного (Labels) индекса в виде чисел и неявные, которые не совпадают с порядковыми номерами, т.е. неявными (Positions) индексами.

Напоминаю, у каждого элемента в Series всегда есть два адреса:

- Неявный (Позиционный) индекс: Порядковый номер элемента (0, 1, 2...), как в обычном списке. Он есть всегда, даже если вы его не видите.
- Явный индекс (Метка/Label): То, что вы задаете сами (строки, даты, числа). Если вы его не задали, он совпадает с неявным.

Пример: есть какие-то числовые данные за конкретные года (года принято указывать в числовом, а не строковом формате)

```
data = [500, 600, 700]
years = [2010, 2011, 2012]
s_years = pd.Series(data, index=years)
print(s_years)
```

Увидим в консоли:

```
2010 500
2011 600
2012 700
dtype: int64
```

При попытке обратиться к первому элементу

```
print(s_years[0])
```

мы получим ошибку `KeyError: 0`. Если явный индекс состоит из целых чисел, pandas при использовании квадратных скобок ищет только явную метку. Он видит, что метки 0 нет (есть только 2010, 2011...), и

Перейдём к основной структуре Pandas. **DataFrame** - это двумерная табличная структура данных. Создаётся DataFrame с помощью конструктора

pd.DataFrame() из **словаря списков, списков словарей** (это самый популярный способ при работе с JSON и API, при котором каждый словарь - это одна строка, а ключи словарей становятся колонками; если у одного из переданных словарей какой-то ключ отсутствует, то pandas автоматически создаёт его и присваивает ему значение `NaN`), **списка списков или массива NumPy** (этот способ чаще всего используют для csv-файлов без заголовков, данные передаются "как есть"/, построчно; имена колонок нужно передать отдельно через параметр `columns`).

В случае создания DataFrame из словаря списков все списки должны быть одинаковой длины, иначе pandas выдаст ошибку `ValueError: All arrays must be of the same length`.

Пример создания DataFrame из словаря списков:

```
import pandas as pd
```

```
data = {
    'Name': ['Anna', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['Moscow', 'London', 'Paris']
}
df = pd.DataFrame(data)
print(df)
```

УВИДИМ В КОНСОЛИ:

Таблица 1.3 – Пример данных

	Name	Age	City
0	Anna	25	Moscow
1	Bob	30	London
2	Charlie	35	Paris

Пример создания DataFrame из списка словарей:

```
data = [
    {'Name': 'Anna', 'Age': 25, 'City': 'Moscow'},
    {'Name': 'Bob', 'Age': 30, 'City': 'London'},
    {'Name': 'Charlie', 'Age': 35}
]

df = pd.DataFrame(data)
print(df)
```

УВИДИМ В КОНСОЛИ:

Таблица 1.4 – Пример данных

	Name	Age	City
0	Anna	25	Moscow
1	Bob	30	London
2	Charlie	35	NaN

Основные атрибуты DataFrame:

- `shape` - показывает, сколько в таблице строк и столбцов;
- `columns` - позволяет получить список всех заголовков;
- `index` - показывает, как занумерованы строки;
- `dtypes` - показывает типы данных в каждой колонке;

1.3 Библиотека Matplotlib / Seaborn

1.4 Библиотека Scikit-learn

ЗАКЛЮЧЕНИЕ